

Laravel 12 で簡単なタスク リストを作ってみる

Laravel の環境設定

Laravel 12 のインストール

```
composer create-project --prefer-dist laravel/laravel tasks "12.*"
```

Laravel Debugbar をインストール

```
cd tasks  
composer require barryvdh/laravel-debugbar --dev  
php artisan vendor:publish --provider="Barryvdh\Debugbar\ServiceProvider"
```

laravel/ui と Bootstrap と Auth をインストール

```
composer require laravel/ui --dev  
php artisan ui bootstrap --auth
```

※ The [Controller.php] file already exists. Do you want to replace it? (yes/no) [no] というような質問が表示された場合は、すべて「yes」と入力して Enter キーを押してください。

```
npm install && npm run build
```

データベースの接続設定

SQLite を使います。

Laravel 12 では、既に database/database.sqlite は作成されていますので、あらためて作成する必要はありません。

.env は、デフォルトで SQLite に接続するようになっていますので、編集する必要はありません。

動作確認

resources/views/welcome.blade.php 15 行目を修正します。

```
@vite(['resources/css/app.css', 'resources/js/app.js'])
```

↓

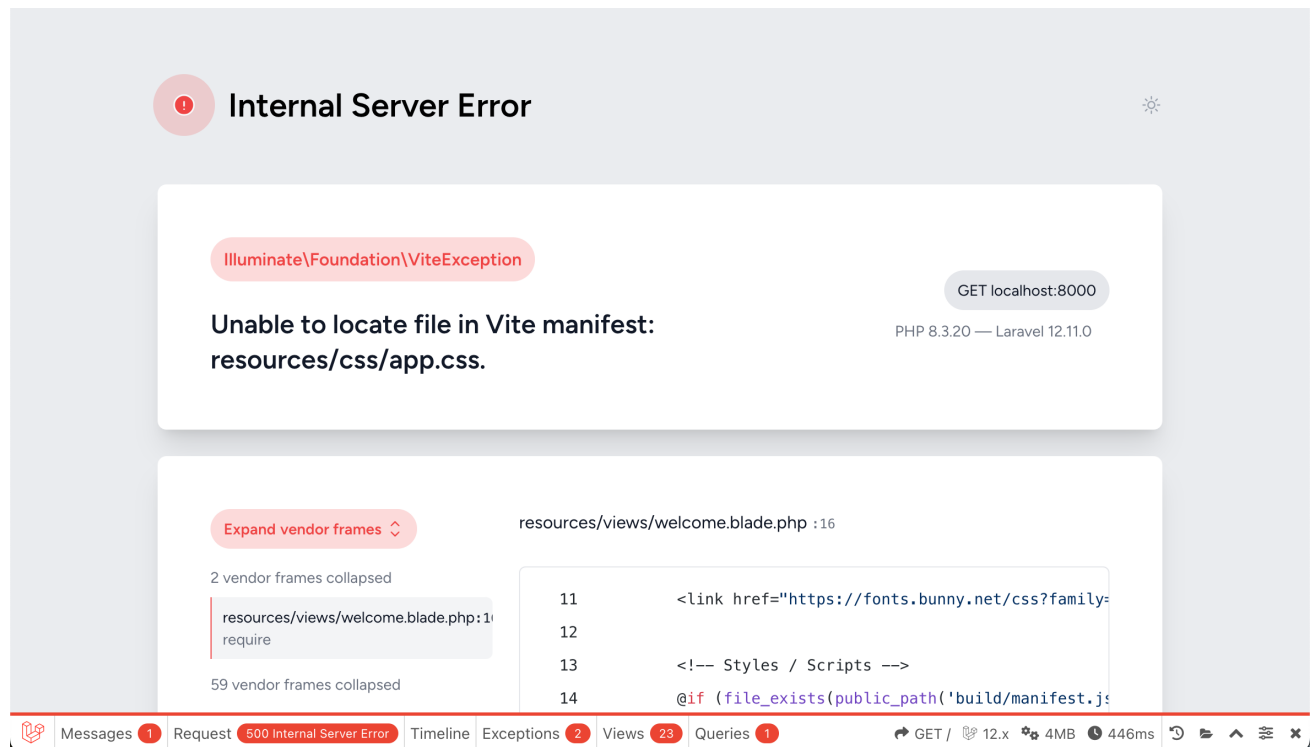
```
@vite(['resources/sass/app.scss', 'resources/js/app.js'])
```

PHP の組み込み http サーバーで動作確認します。

```
php artisan serve
```

<http://localhost:8000> にアクセスします。

エラーが表示されますが、サーバーが起動できれば OK です。



Internal Server Error

Illuminate\Foundation\ViteException

GET localhost:8000

Unable to locate file in Vite manifest:
resources/css/app.css.

PHP 8.3.20 — Laravel 12.11.0

Expand vendor frames

resources/views/welcome.blade.php :16

```
11 <link href="https://fonts.bunny.net/css?family=
12
13 <!-- Styles / Scripts -->
14 @if (file_exists(public_path('build/manifest.js
```

モデルとマイグレーションとシーダー

モデルと、マイグレーションと、シーダーを作成します。

モデルとマイグレーションの作成

```
php artisan make:model Task -m
```

"-m"オプションは、モデルを作成するときに、一緒にマイグレーションも作成してくれます。

出来上がったモデルの内容を確認します。

app/Models/Task.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Task extends Model
{
    use HasFactory;
}
```

マイグレーションの内容を確認します。（ファイル名の"2022_02_22_005632"の部分は実行したタイミングで変わります）

database/migrations/2022_02_22_005632_create_tasks_table.php

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     *
     * @return void
     */
    public function up()
    {
        Schema::create('tasks', function (Blueprint $table) {
            $table->id();
            $table->timestamps();
        });
    }
}
```

```

/**
 * Reverse the migrations.
 *
 * @return void
 */
public function down()
{
    Schema::dropIfExists('tasks');
}
};

```

マイグレーションに作成するカラムを追加していきます。

database/migrations/2022_02_22_005632_create_tasks_table.php

```

public function up()
{
    Schema::create('tasks', function (Blueprint $table) {
        $table->id();
        $table->integer('user_id')->comment('ユーザーID');
        $table->string('title')->comment('タスクのタイトル');
        $table->string('description')->comment('タスクの説明');
        $table->date('registration_date')->comment('登録日');
        $table->date('expiration_date')->comment('期限日');
        $table->date('completion_date')->nullable()->comment('完了日 (null のときは未完了)');
        $table->timestamps();
    });
}

```

マイグレーションを実行します。

```
php artisan migrate
```

シーダーの作成と実行

表示確認のためのレコードを作成するシーダーを作成します。

```
php artisan make:seeder TasksTableSeeder
```

作成されたシーダーを確認します。

database/seeder/TasksTableSeeder.php

```

<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

class TasksTableSeeder extends Seeder

```

```
{
    /**
     * Run the database seeds.
     *
     * @return void
     */
    public function run()
    {
        //
    }
}
```

シーダーに追記します。

php artisan make:seeder TasksTableSeeder

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Str;
use App\Models\Task;

class TasksTableSeeder extends Seeder
{
    /**
     * Run the database seeds.
     *
     * @return void
     */
    public function run()
    {
        for ($i = 0; $i < 20; $i++) {
            Task::create([
                'user_id' => 1,
                'title' => Str::random(10),
                'description' => Str::random(30),
                'registration_date' => date('Y-m-d'),
                'expiration_date' => '2022-02-22',
            ]);
        }
    }
}
```

レコードを 20 件作成しています。完了日は値を指定していません（nullable なので可能）。title と description にはランダムな文字列を指定しています。

また、Str クラスを使用するため、"use Illuminate\Support\Str;"を追記しています。

シーダーを実行するために、DatabaseSeeder.php を編集します。

database/seeder/DatabaseSeeder.php

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;

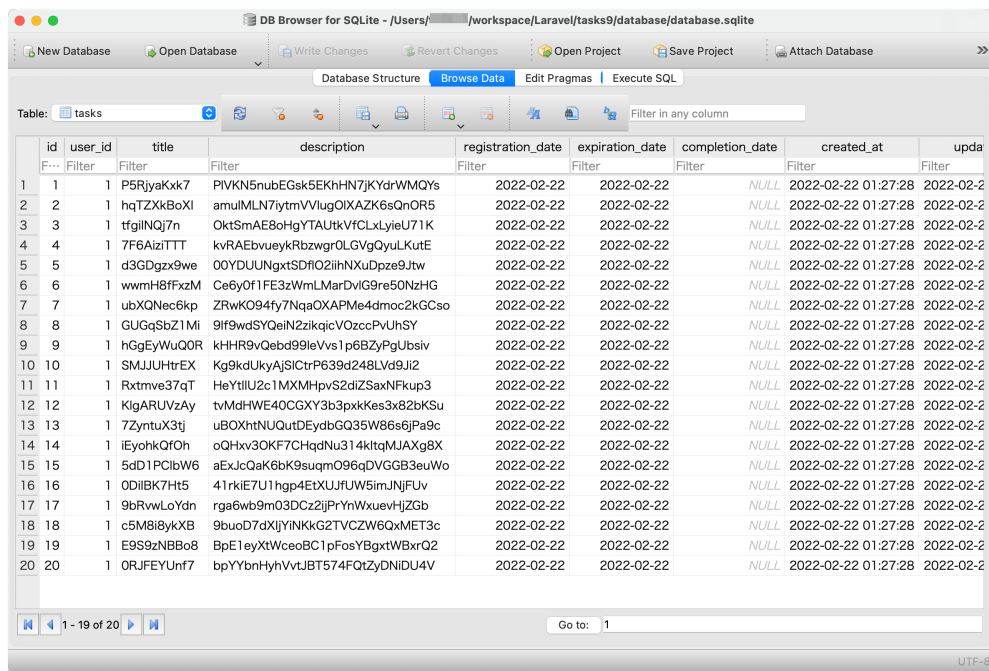
class DatabaseSeeder extends Seeder
{
    /**
     * Seed the application's database.
     *
     * @return void
     */
    public function run()
    {
        // \App\Models\User::factory(10)->create();
        $this->call(TasksTableSeeder::class);
    }
}
```

シーダーを実行します。

```
php artisan db:seed
```

レコードがインサートされているか確認してみます。

DB Browser for SQLite で database/database.sqlite を開いて確認します。



The screenshot shows the DB Browser for SQLite application. The 'tasks' table is selected, and its data is displayed in a table view. The table has 10 columns: id, user_id, title, description, registration_date, expiration_date, completion_date, created_at, and updated_at. There are 20 rows of data, all with a registration_date of 2022-02-22 and an expiration_date of 2022-02-22. The completion_date is NULL for all rows. The created_at and updated_at dates are 2022-02-22 01:27:28 for all rows.

id	user_id	title	description	registration_date	expiration_date	completion_date	created_at	updated_at
1	1	P5RjyaKxk7	PIVKN5nubEGsk5EKHhN7JKYdrWMQYs	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
2	1	hQITZXkBoXI	amuMLN7iytmVvIugOIXAZK6sQnOR5	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
3	1	tfqilINQj7n	OktSmAE8oHgYTAUtkVfCLxLyieU71K	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
4	1	7F6AiziTTT	kvRAEbvueykRbzwgrOLGvGQyulKutE	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
5	1	d3GDgzx9we	00YDUUNgxtSDfiO2ihN XuDpze9Jtw	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
6	1	wwmH8fFxm	Ce6yOf1FE3zWmLMarDvlG9re50NzHG	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
7	1	ubXQNec6kp	ZRwKO94fy7NqaOXAPMe4dmoc2kGCso	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
8	1	GUGqSbZ1Mi	9lf9wdSYQeIN2zikqicVOzccPvUhsY	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
9	1	hGgEyWuQOR	khHR9vQebd99leVvs1p6BZyPgUbsiv	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
10	1	SMJUUhtrEX	Kg9kdUkyAjSICtrP639d248LVd9Ji2	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
11	1	Rxtmve37qT	HeYtlIU2c1MXMHpvS2diZSaxNFkup3	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
12	1	KlgARUVzAy	tvMdHWE40CGXY3b3pxkKes3x82bkSu	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
13	1	7ZyntuX3tj	uBOXhtNUQutDEydbGQ35W866jPa9c	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
14	1	iEyoHkQfOh	oQHxv3OKF7CHqdNu314kittqMJAXg8X	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
15	1	5dD1PClbW6	aExJcQaK6bK9suqmO96qDVGGB3euWo	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
16	1	0DiBK7Ht5	41rkiE7U1hgp4EtXUJfUW5imNJFuv	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
17	1	9bRvwLoYdn	rga6wb9m03DCz2jPrYrnWxuevHJZGb	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
18	1	c5M8i8ykXB	9buoD7dxjYiNKKg2TVCZW6QxMET3c	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
19	1	E9S9zNBBo8	BpE1eyXtWceoBC1pFosYBgxtWBxrQ2	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28
20	1	ORJFEYUnf7	bpYbnHvhVvtJBT574FQtzYDNIDU4V	2022-02-22	2022-02-22	NULL	2022-02-22 01:27:28	2022-02-22 01:27:28

モデルのリレーション

User モデルと Task モデルのリレーションを設定します。

User が複数の Task を持っているので、User から見ると、

- User が 1 に対して Task が多

となるように、リレーションを設定します。下記のように追記します。

app/Models/User.php

```
/**
 * リレーション
 */
public function task()
{
    return $this->hasMany(\App\Models\Task::class);
}
```


ルーティングとログイン後の遷移先

ルーティング

ルーティングの情報を書いておきます。

routes/web.php

```
Route::resource('tasks', App\Http\Controllers\TaskController::class)->middleware('auth');
```

RESTful で作ってみます。ログインしないと使えないようにしておきます。

ログイン後の遷移先

ログイン後とユーザー登録後の遷移先が home になっているので、tasks に変更します。

app/Http/Controllers/Auth/LoginController.php

```
/**
 * Where to redirect users after login.
 *
 * @var string
 */
// protected $redirectTo = RouteServiceProvider::HOME;
protected $redirectTo = 'tasks';
```

app/Http/Controllers/Auth/RegisterController.php

```
/**
 * Where to redirect users after registration.
 *
 * @var string
 */
// protected $redirectTo = RouteServiceProvider::HOME;
protected $redirectTo = 'tasks';
```

コントローラを作成する

コントローラを作成します。" --resource"オプションを追加すると、RESTful なコントローラを作成してくれます。

```
php artisan make:controller TaskController --resource
```

出来上がったコントローラを確認します。

app/Http/Controllers/TaskController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class TaskController extends Controller
{
    /**
     * Display a listing of the resource.
     *
     * @return \Illuminate\Http\Response
     */
    public function index()
    {
        //
    }

    /**
     * Show the form for creating a new resource.
     *
     * @return \Illuminate\Http\Response
     */
    public function create()
    {
        //
    }

    /**
     * Store a newly created resource in storage.
     *
     * @param \Illuminate\Http\Request $request
     * @return \Illuminate\Http\Response
     */
    public function store(Request $request)
    {
        //
    }

    /**
     * Display the specified resource.
     *
     * @param int $id
     * @return \Illuminate\Http\Response
     */
}
```

```

public function show($id)
{
    //
}

/**
 * Show the form for editing the specified resource.
 *
 * @param int $id
 * @return \Illuminate\Http\Response
 */
public function edit($id)
{
    //
}

/**
 * Update the specified resource in storage.
 *
 * @param \Illuminate\Http\Request $request
 * @param int $id
 * @return \Illuminate\Http\Response
 */
public function update(Request $request, $id)
{
    //
}

/**
 * Remove the specified resource from storage.
 *
 * @param int $id
 * @return \Illuminate\Http\Response
 */
public function destroy($id)
{
    //
}
}

```

自動的に作成されたアクションメソッドは、下記のとおりです。

- index() インデックスページ
- create() → store() 新規作成ページ→新規登録処理
- show() 詳細ページ
- edit() → update() 修正ページ→修正処理
- destroy() 削除処理

ベースのレイアウト

ベースとなるレイアウトは、laravel/ui のテンプレートを使います。内容を確認します。

resources/views/layouts/app.blade.php

```
1. <!doctype html>
2. <html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
3. <head>
4.     <meta charset="utf-8">
5.     <meta name="viewport" content="width=device-width, initial-scale=1">
6.
7.     <!-- CSRF Token -->
8.     <meta name="csrf-token" content="{{ csrf_token() }}">
9.
10.    <title>{{ config('app.name', 'Laravel') }}</title>
11.
12.    <!-- Scripts -->
13.    <script src="{{ asset('js/app.js') }}" defer></script>
14.
15.    <!-- Fonts -->
16.    <link rel="dns-prefetch" href="//fonts.gstatic.com">
17.    <link href="https://fonts.googleapis.com/css?family=Nunito" rel="stylesheet">
18.
19.    <!-- Styles -->
20.    <link href="{{ asset('css/app.css') }}" rel="stylesheet">
21. </head>
22. <body>
23.     <div id="app">
24.         <nav class="navbar navbar-expand-md navbar-light bg-white shadow-sm">
25.             <div class="container">
26.                 <a class="navbar-brand" href="{{ url('/') }}">
27.                     {{ config('app.name', 'Laravel') }}
28.                 </a>
29.                 <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded="false" aria-
label="{{ __('Toggle navigation') }}">
30.                     <span class="navbar-toggler-icon"></span>
31.                 </button>
32.
33.                 <div class="collapse navbar-collapse" id="navbarSupportedContent">
34.                     <!-- Left Side Of Navbar -->
35.                     <ul class="navbar-nav me-auto">
36.
37.                     </ul>
38.
39.                     <!-- Right Side Of Navbar -->
40.                     <ul class="navbar-nav ms-auto">
41.                         <!-- Authentication Links -->
42.                         @guest
43.                             @if (Route::has('login'))
44.                                 <li class="nav-item">
45.                                     <a class="nav-link"
href="{{ route('login') }}">{{ __('Login') }}</a>
46.                                 </li>
```

```

47.         @endif
48.
49.         @if (Route::has('register'))
50.             <li class="nav-item">
51.                 <a class="nav-link"
href="{{ route('register') }}">{{ __('Register') }}</a>
52.             </li>
53.         @endif
54.         @else
55.             <li class="nav-item dropdown">
56.                 <a id="navbarDropdown" class="nav-link dropdown-toggle" href="#"
role="button" data-bs-toggle="dropdown" aria-haspopup="true" aria-expanded="false" v-pre>
57.                     {{ Auth::user()->name }}
58.                 </a>
59.
60.                 <div class="dropdown-menu dropdown-menu-end" aria-
labelledby="navbarDropdown">
61.                     <a class="dropdown-item" href="{{ route('logout') }}"
62.                         onclick="event.preventDefault();
63.                             document.getElementById('logout-form').submit();">
64.                         {{ __('Logout') }}
65.                     </a>
66.
67.                     <form id="logout-form" action="{{ route('logout') }}" method="POST"
class="d-none">
68.                         @csrf
69.                     </form>
70.                 </div>
71.             </li>
72.         @endguest
73.     </ul>
74. </div>
75. </div>
76. </nav>
77.
78.     <main class="py-4 container-md ">
79.         @yield('content')
80.     </main>
81. </div>
82. </body>
83. </html>
84.

```

78 行目に"container-md"クラスを追加しています。79 行目の"@yield('content')"のところに各ページのテンプレートを作って埋め込むようにします。

※実際の行数は環境によって異なるかもしれません。

ページを作成する

一覧表示

タスクの一覧を表示するためのテンプレートを作成します。

resources/views/tasks/index.blade.php

```
@extends('layouts.app')

@section('content')

<h1>タスク一覧</h1>
<table class="table">
    <tr>
        <th>登録日</th>
        <th>タイトル</th>
        <th>期限日</th>
        <th>完了日</th>
    </tr>
    @foreach ($tasks as $task)
    <tr>
        <td>{{ $task->registration_date }}</td>
        <td>{{ $task->title }}</td>
        <td>{{ $task->expiration_date }}</td>
        <td>{{ $task->completion_date }}</td>
    </tr>
    @endforeach
</table>

@endsection
```

認証されたユーザーの情報（Auth::user()）を取得するメソッドを使うので、下記をコードに追記しておきます。

app/Http/Controllers/TaskController.php

```
use Illuminate\Support\Facades\Auth;
```

コントローラに、レコードを抽出してテンプレートに渡すコードを記載します。

app/Http/Controllers/TaskController.php

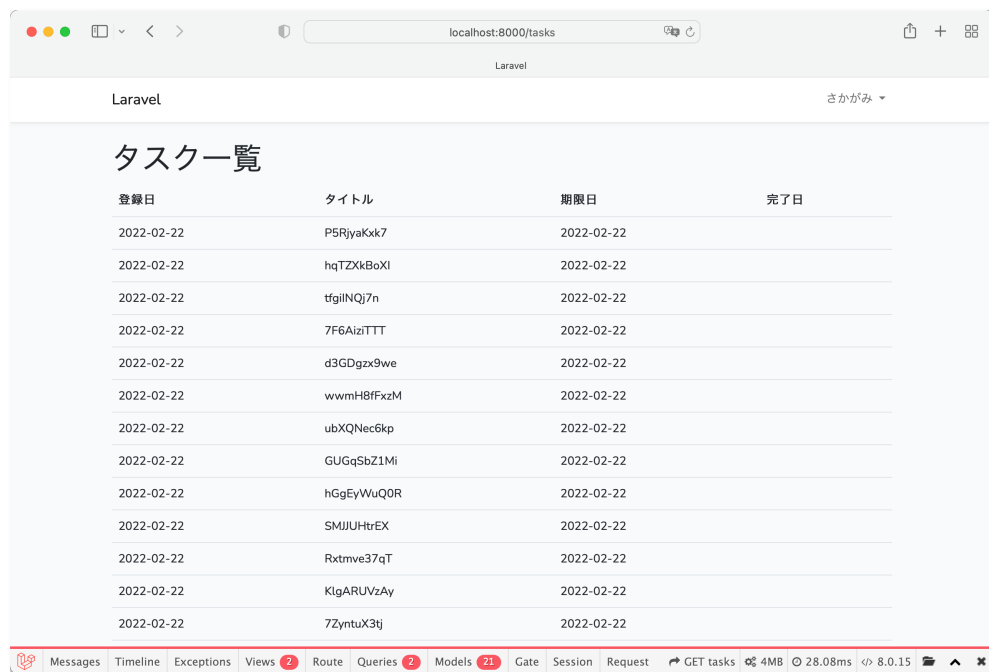
```
public function index()
{
    $tasks = Auth::user()->task()->get();
    return view('tasks.index', compact('tasks'));
}
```

compact()メソッドについては、下記の Web ページを参照してください。

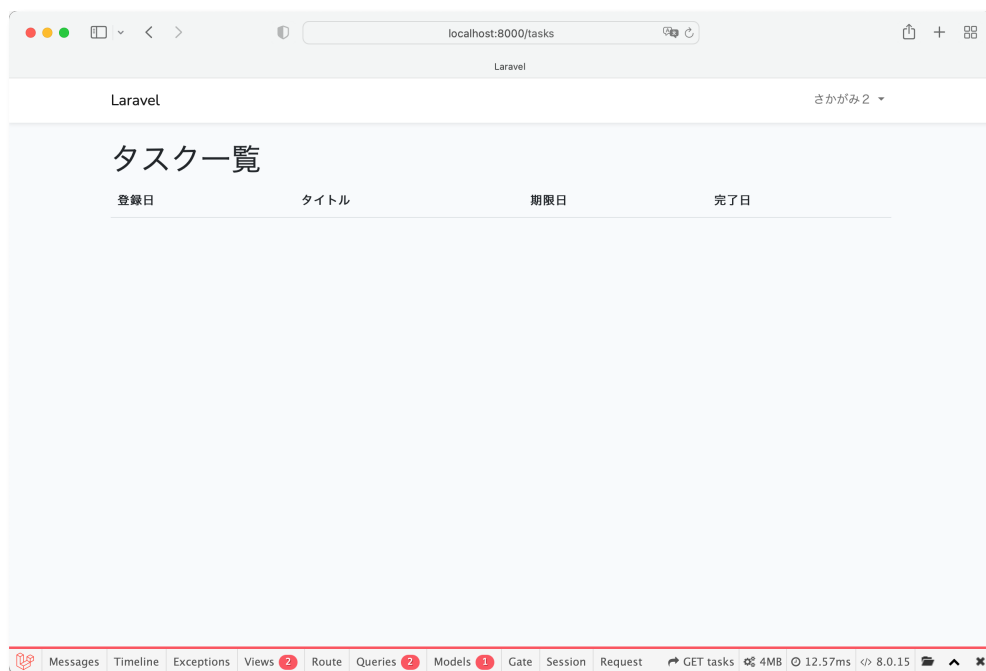
compact — 変数名とその値から配列を作成する

<https://www.php.net/manual/ja/function.compact.php>

ユーザー登録すると、次の画面に遷移します。



別のユーザーで登録してログインすると、何もレコードが表示されないはずです。



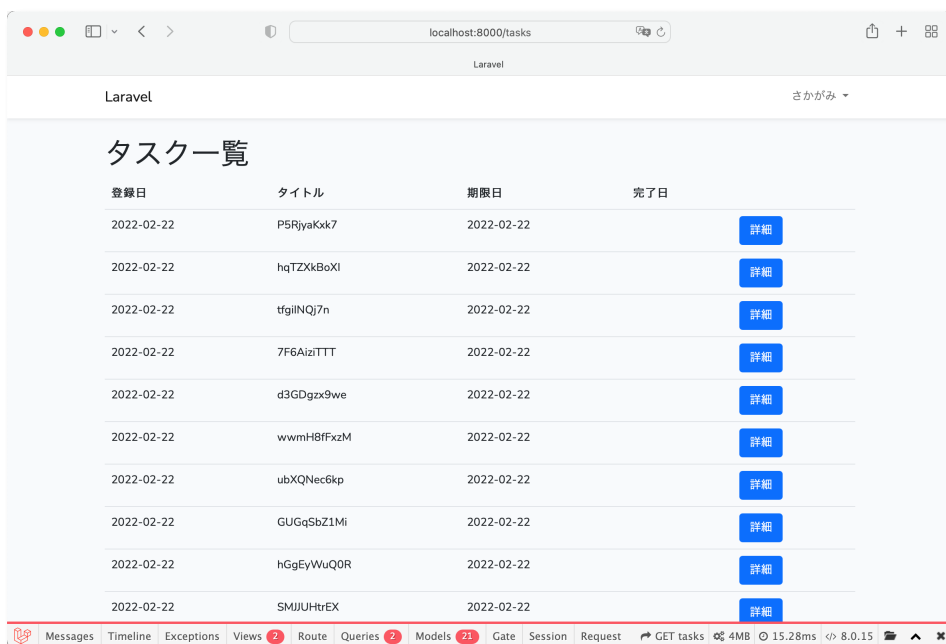
詳細表示

一覧表示ページに、詳細表示ページへのリンクボタンを追加します。

resources/views/tasks/index.blade.php

```
1. @extends('layouts.app')
2.
3. @section('content')
4.
5. <h1>タスク一覧</h1>
6. <table class="table">
7.     <tr>
8.         <th>登録日</th>
9.         <th>タイトル</th>
10.        <th>期限日</th>
11.        <th>完了日</th>
12.        <th></th>
13.    </tr>
14.    @foreach ($tasks as $task)
15.        <tr>
16.            <td>{{ $task->registration_date }}</td>
17.            <td>{{ $task->title }}</td>
18.            <td>{{ $task->expiration_date }}</td>
19.            <td>{{ $task->completion_date }}</td>
20.            <td>
21.                <a href="{{ route('tasks.show', $task) }}" class="btn btn-primary">詳細</a>
22.            </td>
23.        </tr>
24.    @endforeach
25. </table>
26.
27. @endsection
28.
```

20 行目にリンクボタンを追加しています。ブラウザで確認したところ。「詳細」ボタンが追加されました。



コントローラにコードを記述します。

"http://localhost:8000/tasks/2"のように、URL の後ろにタスクの ID が追加されます。URL の ID を書き換えると他のユーザーのタスクが見えてしまいます。そこで、タスクの ID をチェックし、ユーザーID とタスク ID が異なるときには HTTP レスポンスステータス 404 を返却 (HttpException をスロー) するメソッドをコントローラに作ります。

abort()メソッドを使います。

一番下に記述すればいいでしょう。

app/Http/Controllers/TaskController.php

```
// クラス宣言の前で追記しておく
use App\Models\Task;

/**
 * ログインユーザーID とタスクのユーザーID が異なるときに HttpException をスローする
 *
 * @param Task $task
 * @param integer $status
 * @return void
 */
private function checkUserID(Task $task, int $status = 404)
{
    // ログインユーザーID とタスクのユーザーID が異なるとき
    if (Auth::user()->id != $task->user_id) {
        // HTTP レスポンスステータスコードを返却
        abort($status);
    }
}
```

show()メソッドで checkUser()メソッドを使います。

app/Http/Controllers/TaskController.php

```
/**
 * Display the specified resource.
 *
 * @param Task $task
 * @return \Illuminate\Http\Response
 */
public function show(Task $task)
{
    $this->checkUserID($task);
    return view('tasks.show', compact('task'));
}
```

詳細表示のテンプレートを作成します。

resources/views/tasks/show.blade.php

```
1. @extends('layouts.app')
2.
3. @section('content')
```

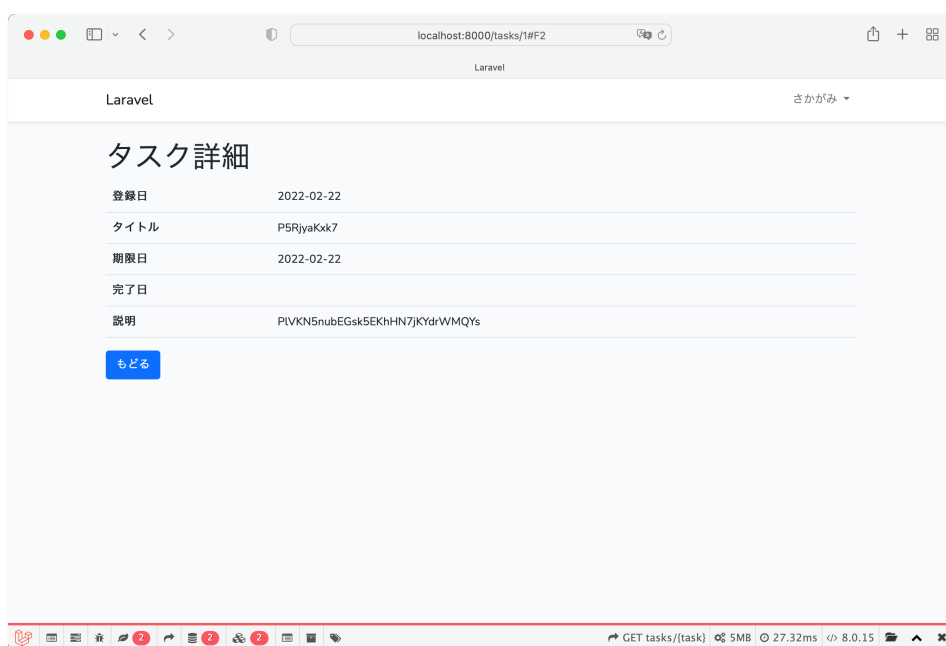
```

4.
5. <h1>タスク詳細</h1>
6. <table class="table">
7.     <tr>
8.         <th>登録日</th>
9.         <td>{{ $task->registration_date }}</td>
10.    </tr>
11.    <tr>
12.        <th>タイトル</th>
13.        <td>{{ $task->title }}</td>
14.    </tr>
15.    <tr>
16.        <th>期限日</th>
17.        <td>{{ $task->expiration_date }}</td>
18.    </tr>
19.    <tr>
20.        <th>完了日</th>
21.        <td>{{ $task->completion_date }}</td>
22.    </tr>
23.    <tr>
24.        <th>説明</th>
25.        <td>{!! nl2br(e($task->description)) !!}</td>
26.    </tr>
27. </table>
28. <a href="{{ route('tasks.index') }}" class="btn btn-primary">もどる</a>
29.
30. @endsection
31.

```

25 行目、"description"は、テキストエリアで入力する予定なので、nl2br()で改行コードを改行タグに変換して います。ところが、改行タグ自体もエスケープ処理されてしまうので、e()でエスケープ処理してから nl2br()で改行コードを改行タグに変換し、その出力自体はエスケープ処理しないようにしています。

次のように表示されれば OK です。



新規追加処理

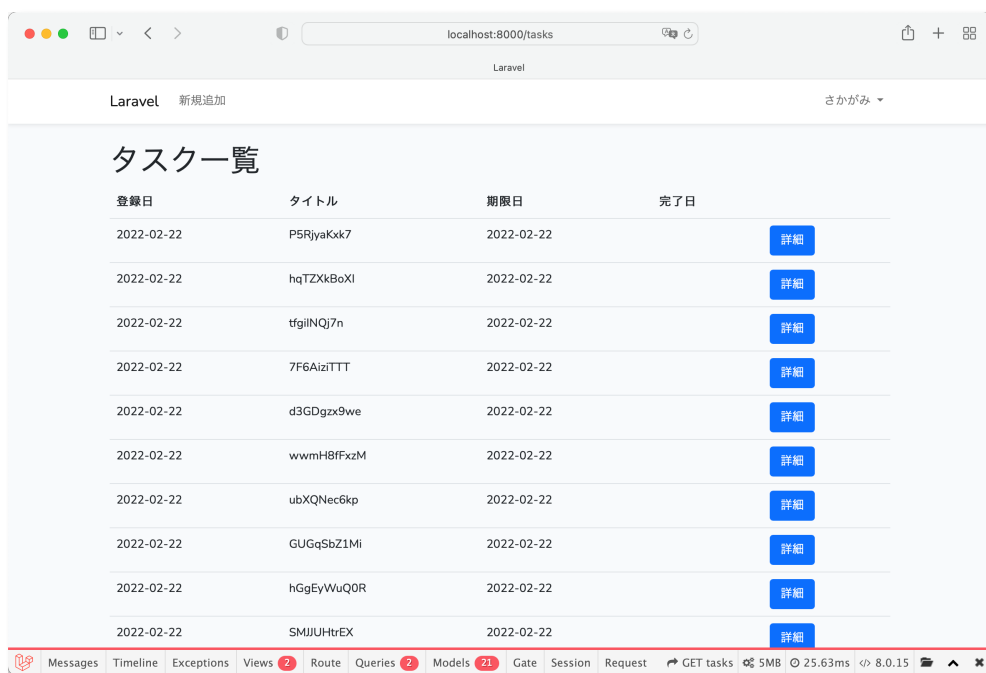
タスクの新規追加処理を作成します。

ナビに、新規追加ページへのリンクを追加します。

resources/views/layouts/app.blade.php

```
33.         <div class="collapse navbar-collapse" id="navbarSupportedContent">
34.             <!-- Left Side Of Navbar -->
35.             <ul class="navbar-nav me-auto">
36.                 @auth
37.                 <li class="nav-item">
38.                     <a href="{{route('tasks.create')}}" class="nav-link">新規追加</a>
39.                 </li>
40.                 @endauth
41.             </ul>
42.
```

36 行目から 40 行目にリンクを追加しました。次のように表示されます。



新規追加ページのテンプレートを作成します。

resources/views/tasks/create.blade.php

```
@extends('layouts.app')

<style>
    input[type="date"] {
        width: 200px;
    }
</style>
```

```

@section('content')

<h1>タスクを新規追加</h1>
<form action="{{ route('tasks.store') }}" method="post">
    @csrf
    <div class="mb-3">
        <label for="title">タイトル</label>
        <input type="text" name="title" id="title" class="form-control" value="{{ old('title') }}">
    </div>
    <div class="mb-3">
        <label for="expiration_date">期限日</label>
        <input type="date" name="expiration_date" id="expiration_date" class="form-control"
            value="{{ old('expiration_date') }}">
    </div>
    <div class="mb-3">
        <label for="completion_date">完了日</label>
        <input type="date" name="completion_date" id="completion_date" class="form-control"
            value="{{ old('completion_date') }}">
    </div>
    <div class="mb-3">
        <label for="description">説明</label>
        <textarea name="description" id="description" rows="5" class="form-
control">{{ old('description') }}</textarea>
    </div>
    <button class="btn btn-primary" type="submit">送信</button>
    <a href="{{ route('tasks.index') }}" class="btn btn-secondary">戻る</a>
</form>

@endsection

```

各コントロールの value 属性は、後でバリデーションを組み込むときのために、old() メソッドを使って、元の送信値が
入力されるようにしています。

コントローラに、新規追加ページを表示するためのコードを記述します。

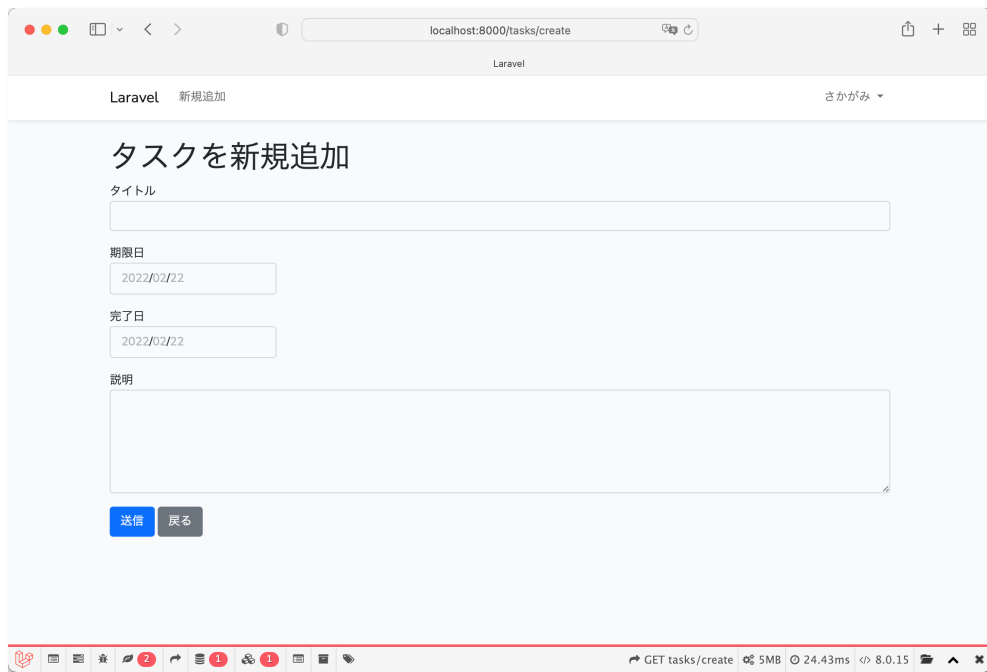
app/Http/Controllers/TaskController.php

```

/**
 * Show the form for creating a new resource.
 *
 * @return \Illuminate\Http\Response
 */
public function create()
{
    return view('tasks.create');
}

```

一覧表示の「新規追加」ボタンをクリックすると、次のページが表示されます。



次に、コントローラにレコードをインサートするコードを記述するのですが、その前に、モデルに

- `$guarded` → 値を反映をしたくないフィールドを配列で指定する
- `$fillable` → 値を反映したいフィールドを配列で指定する

というプロパティを設定する必要があります。どちらかを設定しておく、モデルのプロパティに値を設定するときに `fill()` メソッドが使えるので便利です。

app/Task.php

```
/** @var array 値を代入しないプロパティ */
protected $guarded = [
    'id',
    'user_id',
];

// /** @var array 値を代入するプロパティ */
// protected $fillable = [
//     'title',
//     'description',
//     'registration_date',
//     'expiration_date',
//     'completion_date',
// ];
```

`$guarded` と `$fillable` のどちらかだけを設定しておけばいいので、`$guarded` だけ設定しておきました。

'user_id'が`$guarded`に加えてあるのは、タスクのユーザーIDを他のユーザーIDに書き換えられるのを防ぐためです。という意味合いにおいては、`$guarded`に「値を反映させたくないフィールドを設定する」ということが、セキュリティ上も都合がいい、ということになりますね。

コントローラに新規レコード追加のコードを記述します。

app/Http/Controllers/TaskController.php

```
/**
 * Store a newly created resource in storage.
 *
 * @param \Illuminate\Http\Request $request
 * @return \Illuminate\Http\Response
 */
public function store(Request $request)
{
    $task = new Task();
    $task->fill($request->all());
    // ログインユーザーID
    $task->user_id = Auth::user()->id;
    // 登録日は現在の年月日
    $task->registration_date = date('Y-m-d');
    $task->save();

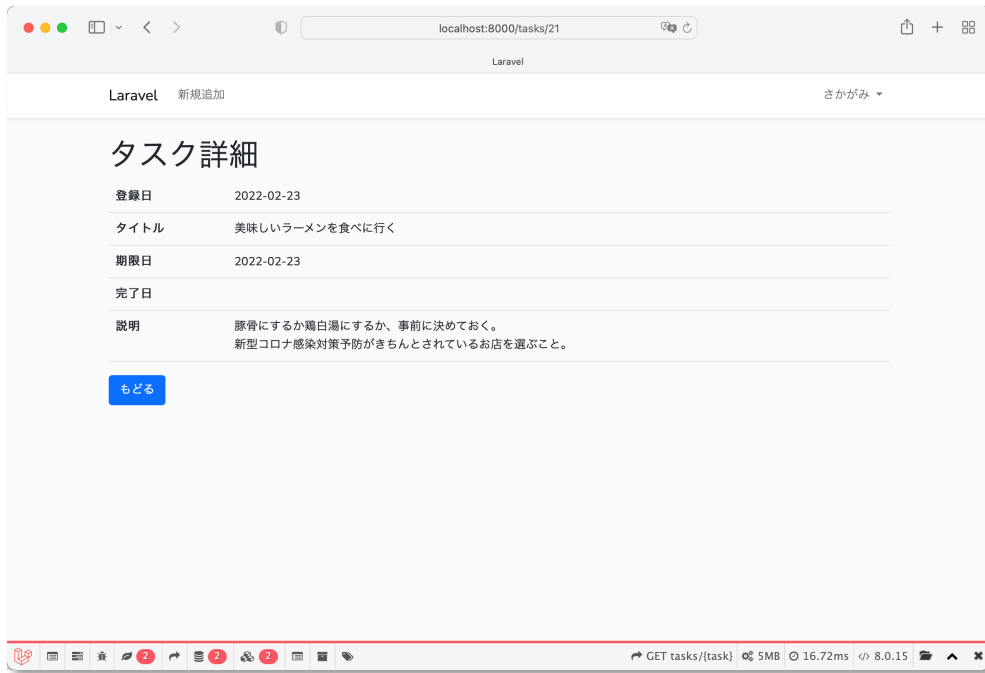
    return redirect()->route('tasks.show', $task);
}
```

書籍によっては CSRF のトークンを unset() しているものがありますが、その必要はありません。

フォームから POST されてくる項目は \$request->all() で取得できます。モデルで \$guarded か \$fillable のどちらかが設定されていないと、fill() メソッドは使えません。

ログインユーザーID と登録日はフォームから POST されて来ないので、個別に値を代入しています。

POST すると、詳細ページにリダイレクトして、登録された内容が表示されます。



更新処理

タスクの更新処理を作成します。

一覧表示ページに、更新ページへのリンクボタンを追加します。

resources/views/tasks/index.blade.php

```

1.  @extends('layouts.app')
2.
3.  @section('content')
4.
5.  <h1>タスク一覧</h1>
6.  <table class="table">
7.      <tr>
8.          <th>登録日</th>
9.          <th>タイトル</th>
10.         <th>期限日</th>
11.         <th>完了日</th>
12.         <th></th>
13.     </tr>
14.     @foreach ($tasks as $task)
15.         <tr>
16.             <td>{{ $task->registration_date }}</td>
17.             <td>{{ $task->title }}</td>
18.             <td>{{ $task->expiration_date }}</td>
19.             <td>{{ $task->completion_date }}</td>
20.             <td>
21.                 <a href="{{ route('tasks.show', $task) }}" class="btn btn-primary">詳細</a>
22.                 <a href="{{ route('tasks.edit', $task) }}" class="btn btn-warning">修正</a>
23.             </td>
24.         </tr>
25.     @endforeach

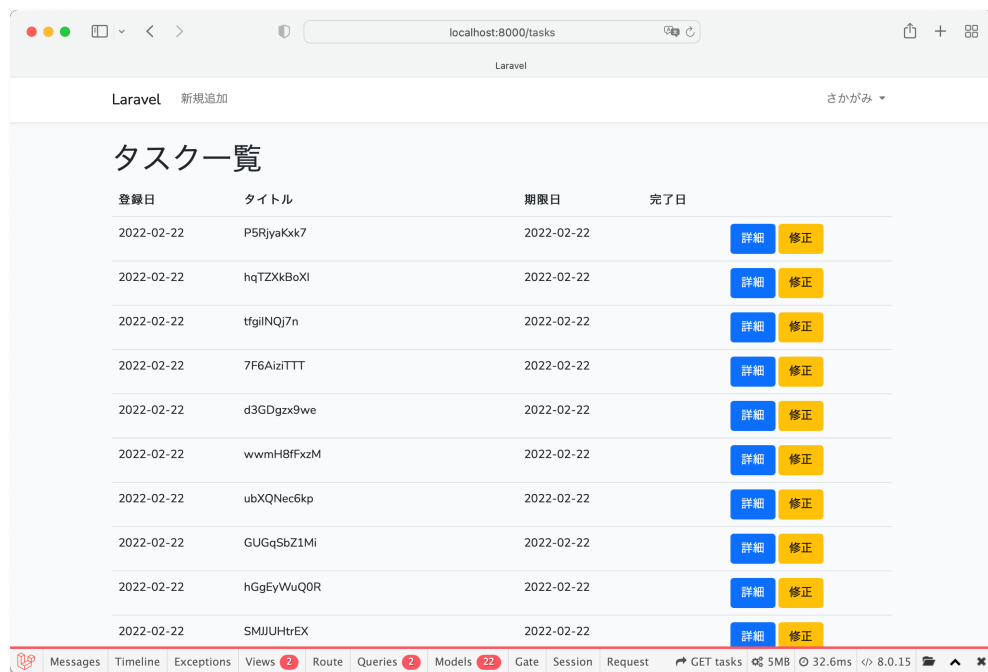
```

```

26. </table>
27.
28. @endsection
29.

```

22 行目に追加しました。このように表示されれば OK です。



編集ページのテンプレートを作成します。create.blade.php をコピーして edit.blade.php にリネームして編集すると楽ですね。

resources/views/tasks/edit.blade.php

```

1. @extends('layouts.app')
2.
3. <style>
4.     input[type="date"] {
5.         width: 200px;
6.     }
7. </style>
8.
9. @section('content')
10.
11. <h1>タスクを修正</h1>
12. <form action="{{ route('tasks.update', $task) }}" method="post">
13.     @csrf
14.     @method('PATCH')
15.     <div class="mb-3">
16.         <label for="title">タイトル</label>
17.         <input type="text" name="title" id="title" class="form-control" value="{{ old('title', $task-
>title) }}">
18.     </div>
19.     <div class="mb-3">
20.         <label for="expiration_date">期限日</label>
21.         <input type="date" name="expiration_date" id="expiration_date" class="form-control "

```



```

22.         value="{{ old('expiration_date', $task->expiration_date) }}">
23.     </div>
24.     <div class="mb-3">
25.         <label for="completion_date">完了日</label>
26.         <input type="date" name="completion_date" id="completion_date" class="form-control"
27.             value="{{ old('completion_date', $task->completion_date) }}">
28.     </div>
29.     <div class="mb-3">
30.         <label for="description">説明</label>
31.         <textarea name="description" id="descriptin" rows="5"
32.             class="form-control ">{{ old('description', $task->description) }}</textarea>
33.     </div>
34.     <button class="btn btn-primary" type="submit">送信</button>
35.     <a href="{{ route('tasks.index') }}" class="btn btn-secondary">戻る</a>
36. </form>
37.
38. @endsection
39.

```

修正の場合は POST メソッドではなく PATCH というメソッドを使うのですが、ブラウザでは PATCH メソッドが使えません。擬似的に PATCH メソッドを使うために、14 行目に"@method('PATCH')"を記述しておきます。

コントローラに更新ページへ遷移するためのコードを記述します。

このコードで該当のモデルをコントローラで取得して、ビューに渡せるわけです。

app/Http/Controllers/TaskController.php

```

/**
 * Show the form for editing the specified resource.
 *
 * @param  \App\Models\Task $task
 * @return \Illuminate\Http\Response
 */
public function edit(Task $task)
{
    $this->checkUserID($task);
    return view('tasks.edit', compact('task'));
}

```

該当のモデルを抽出する処理が抜けているように思われますが、自動的に、仮引数の \$task に該当の ID のモデルが代入されています。この機能のことを "Implicit Binding" (暗黙的なモデルとの結合) といいます。

ビューに Implicit Binding されたモデルを渡すことで、ビューで該当のモデルが持っている値を利用できます。

show() メソッドと同じように、ログインユーザー ID とタスクのユーザー ID が異なるときには、HTTP レスポンスステータス 404 を返却するようにしています。

続けて、update() メソッドを記述します。

app/Http/Controllers/TaskController.php

```

/**
 * Update the specified resource in storage.
 *
 * @param  \Illuminate\Http\Request $request

```

```

* @param \App\Models\Task $task
* @return \Illuminate\Http\Response
*/
public function update(Request $request, Task $task)
{
    $this->checkUserID($task);
    $task->fill($request->all())->save();
    return redirect()->route('tasks.show', $task);
}

```

内容を修正して、送信ボタンをクリックします。

次のように、内容が修正されていれば OK です。

削除処理

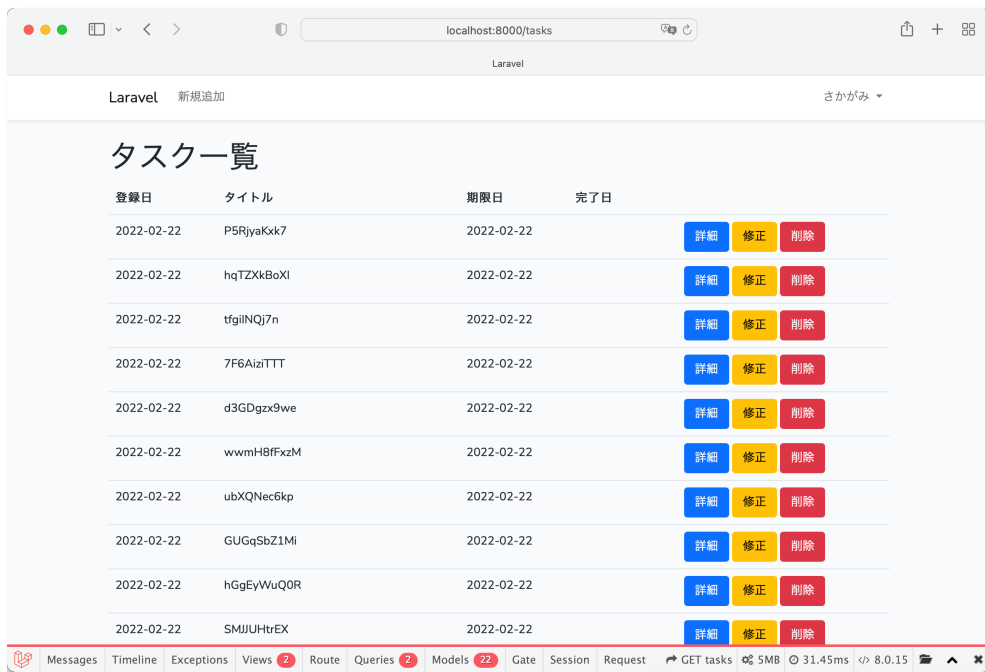
削除処理は画面がないので、一覧表示ページから直接削除できるようにします。一覧表示ページに削除ボタンを追加します。

resources/views/tasks/index.blade.php

```
1. @extends('layouts.app')
2.
3. @section('content')
4.
5. <h1>タスク一覧</h1>
6. <table class="table">
7.     <tr>
8.         <th>登録日</th>
9.         <th>タイトル</th>
10.        <th>期限日</th>
11.        <th>完了日</th>
12.        <th></th>
13.    </tr>
14.    @foreach ($tasks as $task)
15.        <tr>
16.            <td>{{ $task->registration_date }}</td>
17.            <td>{{ $task->title }}</td>
18.            <td>{{ $task->expiration_date }}</td>
19.            <td>{{ $task->completion_date }}</td>
20.            <td>
21.                <form action="{{ route('tasks.destroy', $task) }}" method="post" class="form-inline">
22.                    @method('DELETE')
23.                    @csrf
24.                    <a href="{{ route('tasks.show', $task) }}" class="btn btn-primary mr-2">詳細</a>
25.                    <a href="{{ route('tasks.edit', $task) }}" class="btn btn-warning mr-2">修正</a>
26.                    <button type="submit" class="btn btn-danger" onclick="return confirm('本当に削除します
    か?');">削除</button>
27.                </form>
28.            </td>
29.        </tr>
30.    @endforeach
31. </table>
32.
33. @endsection
34.
```

削除は DELETE メソッドを使うので、form タグを追加しました。

次のように表示されます。



削除ボタンをクリックすると、JavaScript がダイアログボックスを表示します。



「OK」をクリックすると、DELETE メソッドで送信されます。「キャンセル」をクリックすると、何も起きません。コントローラに削除メソッドを記述します。

app/Http/Controllers/TaskController.php

```
/**
 * Remove the specified resource from storage.
 *
 * @param \App\Models\Task $task
 * @return \Illuminate\Http\Response
 */
public function destroy(Task $task)
{
    $this->checkUserID($task);
    $task->delete();
    return redirect()->route('tasks.index');
}
```

レコードが物理削除されて、一覧表示ページに戻ります。

追加と微調整

見栄えと使い勝手を少し調整してみます。

ペジネーションと並び替え

タスクをどんどん追加していくと、一覧表示ページが下へ長くなってしまいますので、ページャーをつけます。また、完了日、期限日、登録日が、それぞれ古い日付から新しい日付へ並べます。コントローラの `index()` メソッドを書き換えます。

app/Http/Controllers/TaskController.php

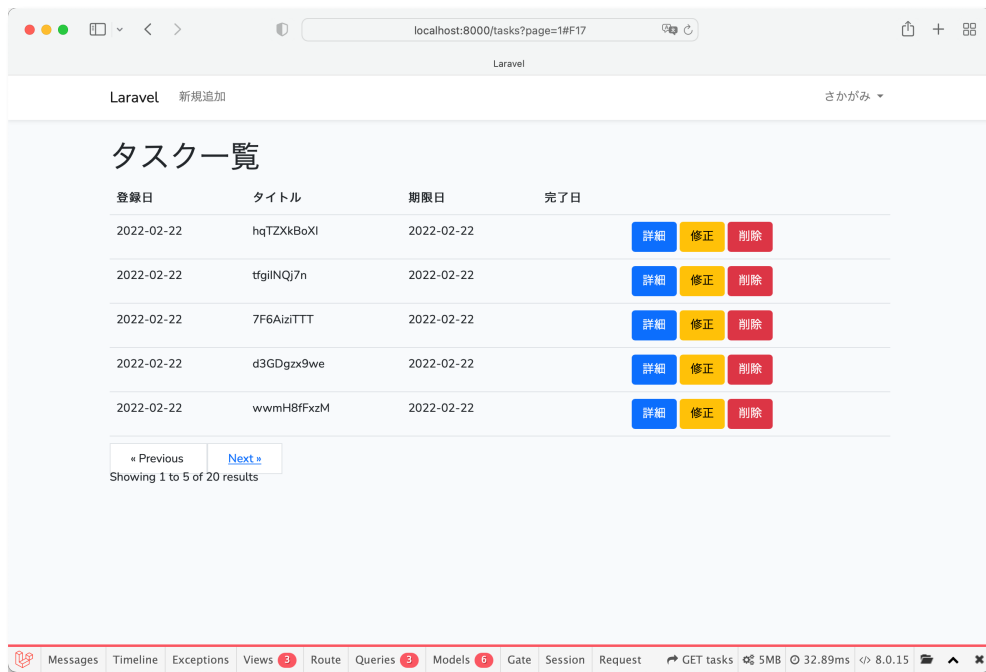
```
/**
 * Display a listing of the resource.
 *
 * @return \Illuminate\Http\Response
 */
public function index()
{
    $tasks = Auth::user()->task()
        ->orderBy('completion_date', 'asc')
        ->orderBy('expiration_date', 'asc')
        ->orderBy('registration_date', 'asc')
        ->paginate(5);
    return view('tasks.index', compact('tasks'));
}
```

テンプレートにページャーを記載します。</table>の下で OK です。

resources/views/tasks/index.blade.php

```
31. </table>
32.
33. {{ $tasks->links() }}
34.
35. @endsection
```

次のように表示されます。



Laravel 12 では、ページャーの装飾にデフォルトで Bootstrap が使われていません。ページャーを Bootstrap で装飾するためには、app/Providers/AppServiceProvider.php に下記のコードを追加します。

app/Providers/AppServiceProvider.php

```
<?php

namespace App\Providers;

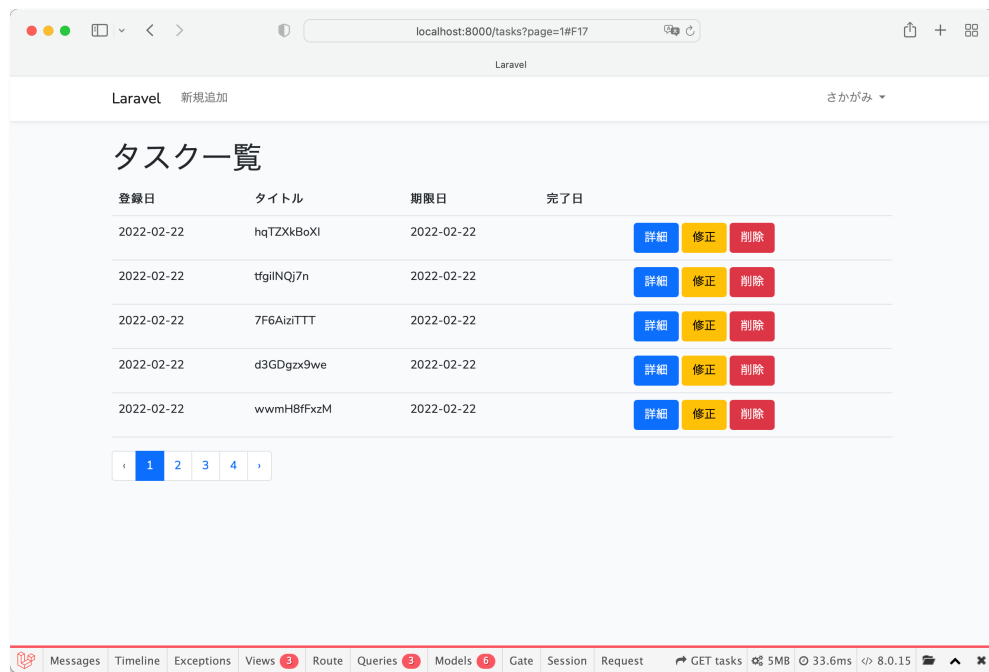
use Illuminate\Pagination\Paginator; // 追記
use Illuminate\Support\ServiceProvider;

class AppServiceProvider extends ServiceProvider
{
    /**
     * Register any application services.
     *
     * @return void
     */
    public function register()
    {
        //
    }

    /**
     * Bootstrap any application services.
     *
     * @return void
     */
    public function boot()
    {
        // ページャーで Bootstrap を使う
        Paginator::useBootstrapFour();
    }
}
```

```
}
```

次のように表示されれば OK です。



アプリケーション名を変える

"Laravel"になっているアプリケーション名を"タスクリスト"に変更します。

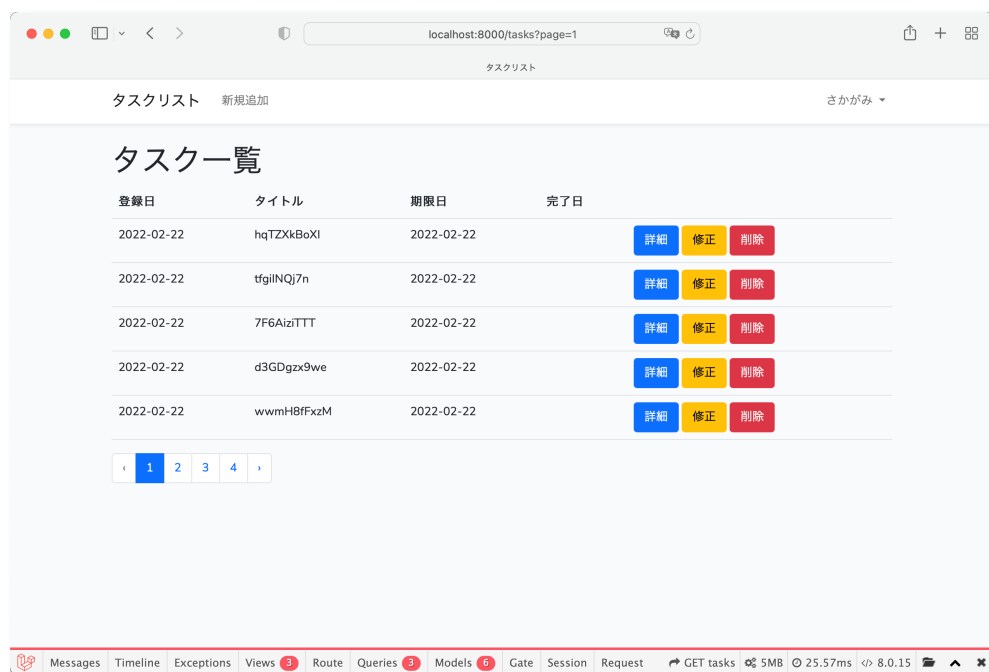
.env

```
APP_NAME=タスクリスト
```

ブラウザをリロードしても表示が変わらないときは、下記のコマンドでキャッシュをクリアします。

```
php artisan config:cache
```

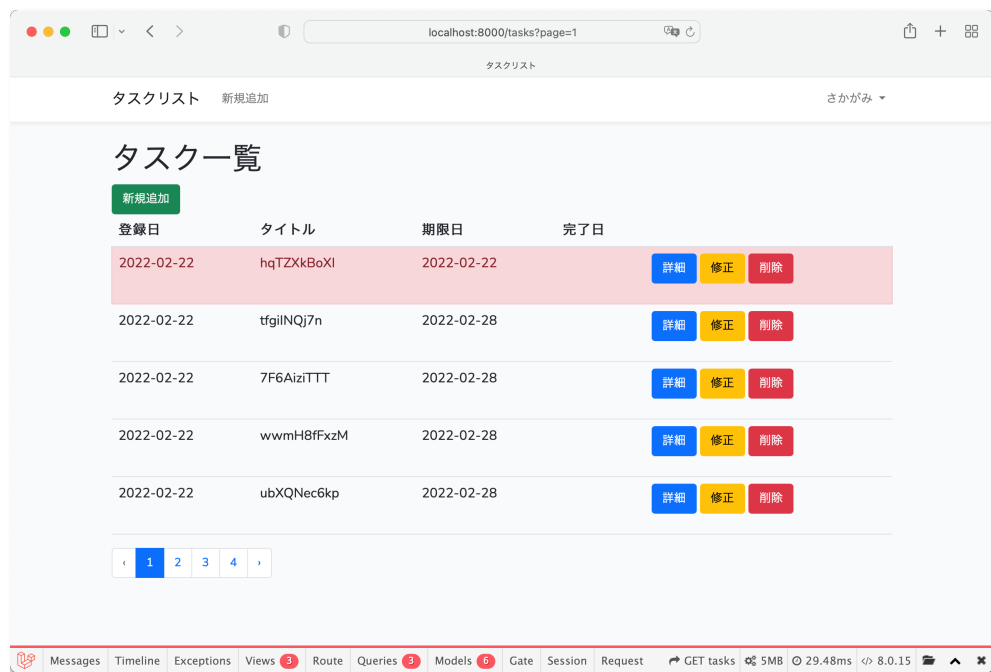
次のように、タイトルが変わります。



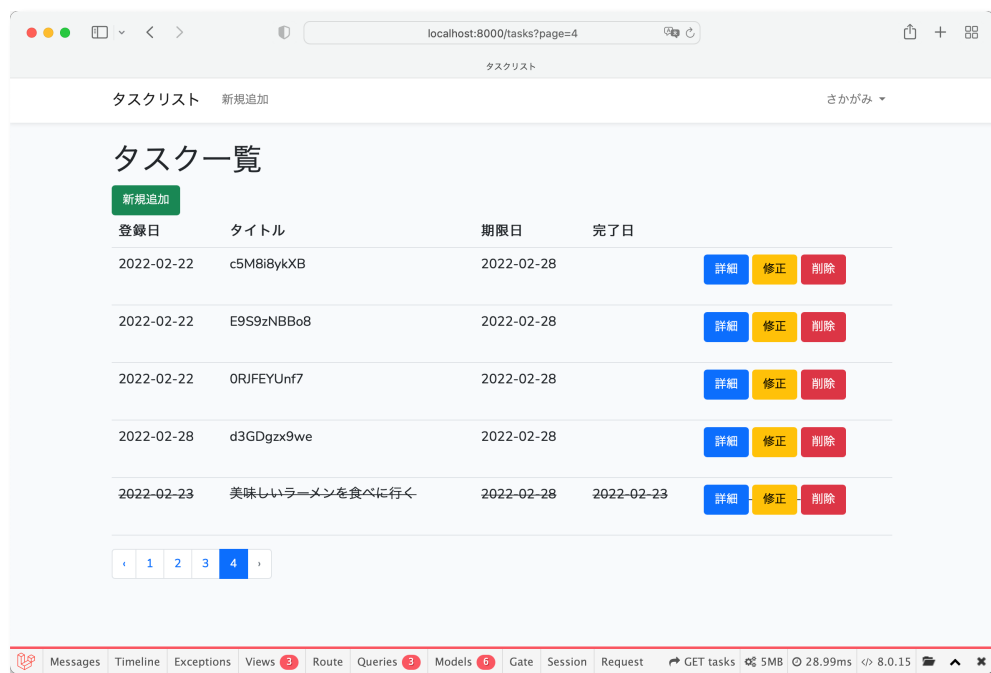
期限が過ぎているタスクは背景を赤くして、完了しているタスクは文字に打ち消し線を入れる

わかりやすい表示に修正します。

終わっていないタスクは背景色をわかりやすい色にします。



また、完了したタスクには、「打ち消し線」を入れます。



resources/views/tasks/index.blade.php

```
1. @extends('layouts.app')
2.
3. @section('content')
```

```

4.
5. <h1>タスク一覧</h1>
6. <table class="table">
7.   <tr>
8.     <th>登録日</th>
9.     <th>タイトル</th>
10.    <th>期限日</th>
11.    <th>完了日</th>
12.    <th></th>
13.  </tr>
14.  @foreach ($tasks as $task)
15.    <tr class="
16.      @if (!is_null($task->completion_date ))
17.        text-decoration-line-through
18.      @elseif ($task->expiration_date < date('Y-m-d'))
19.        table-danger
20.      @endif
21.    ">
22.    <td>{{ $task->registration_date }}</td>
23.    <td>{{ $task->title }}</td>
24.    <td>{{ $task->expiration_date }}</td>
25.    <td>{{ $task->completion_date }}</td>
26.    <td>
27.      <form action="{{ route('tasks.destroy', $task) }}" method="post" class="form-inline">
28.        @method('DELETE')
29.        @csrf
30.        <a href="{{ route('tasks.show', $task) }}" class="btn btn-primary mr-2">詳細</a>
31.        <a href="{{ route('tasks.edit', $task) }}" class="btn btn-warning mr-2">修正</a>
32.        <button type="submit" class="btn btn-danger" onclick="return confirm('本当に削除します
    か？');">削除</button>
33.      </form>
34.    </td>
35.  </tr>
36.  @endforeach
37. </table>
38.
39. {{ $tasks->links() }}
40.
41. @endsection

```

17～21 行目で、条件に応じてクラスを適用しています。

日付をフォーマットする

日付の表示が"2022-02-28"のように"Y-m-d"の形式になっているものを、"2021 年 02 月 28 日"のように"Y 年 m 月 d 日"に変えてみます。

日付形式のフォーマットを変えるには、Date 型もしくは Datetime 型に設定したいプロパティをモデルで指定します。Task モデルに下記のコードを追記します。

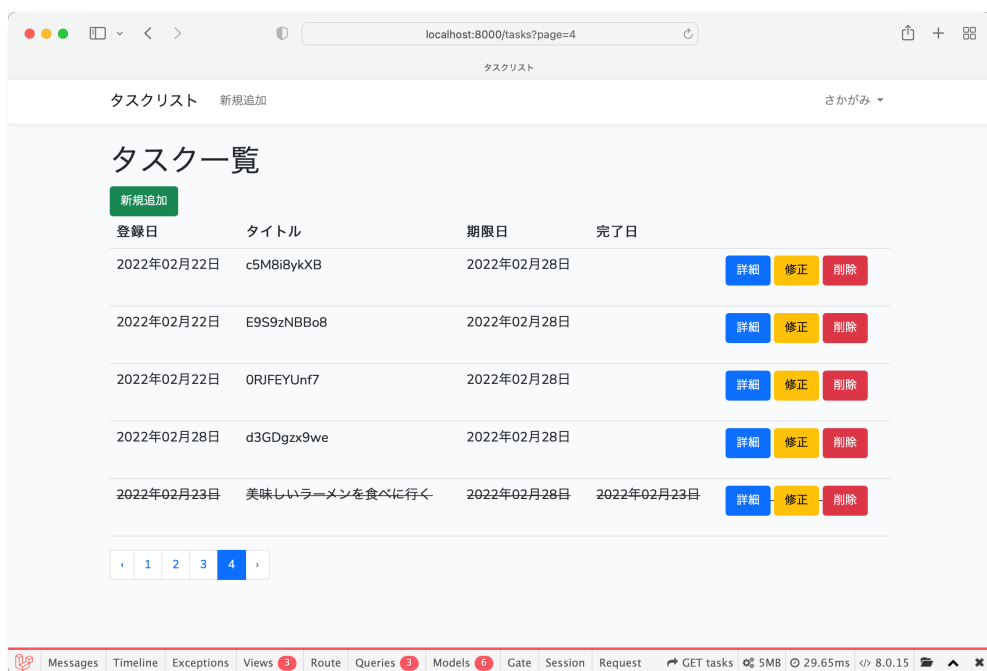
src/tasks/app/Task.php

```
/** @var array 値をキャストするカラム */
protected $casts = [
    'registration_date' => 'date',
    'expiration_date' => 'date',
    'completion_date' => 'date',
];
```

Blade で format() 関数を使って、日付形式のモデルのプロパティをフォーマットします。

```
<td>{{ $task->registration_date->format('Y 年 m 月 d 日') }}</td>
<td>{{ $task->title }}</td>
<td>{{ $task->expiration_date->format('Y 年 m 月 d 日') }}</td>
<td>{{ !is_null($task->completion_date) ? $task->completion_date->format('Y 年 m 月 d 日') :
'' }}</td>
```

"\$task->completion_date"は null 許容になっています。そのため、値が null のときには format() でエラーが起きます。null のときには何も表示しないようにしています。



日付形式のフォーマットを変更できるようにしたことで、日付形式に設定したモデルのプロパティをそのまま Blade で出力すると"Y-m-d H:i:s"の形式になります。そのまま<input type="date">の value 値に設定をするとデフォルト値としてテキストボックスに日付の表示ができないので、フォーマットを変更する必要があります。

下記のようにしてフォーマットを変更した値を value 値に設定します。

resources/views/tasks/edit.blade.php

```
<input type="date" name="expiration_date" id="expiration_date" class="form-control" value="{{ old('expiration_date', $task->expiration_date->format("Y-m-d")) }}">
```

```
<input type="date" name="completion_date" id="completion_date" class="form-control" value="{{ old('completion_date', is_null($task->completion_date) ? "" : $task->completion_date->format("Y-m-d")) }}">
```

バリデーション

バリデーションを書く場所

バリデーションの処理を組み込みます。

Laravel でバリデーションを実行する場所としては、

- コントローラ
- フォームリクエスト

があります。（バリデーションの設定自体はモデルにも記述できますね）

コントローラに様々な処理を書いていくのは簡単でいいのですが、コントローラの行数が増えすぎて肥大化してしまい、コードが読みにくくなるというデメリットがあります。

今回はフォームリクエストにバリデーションの処理を書いて、コントローラで呼び出すようにしていきます。

FormRequest を artisan で作成する

下記のコマンドでフォームリクエストの雛形を作成します。

```
php artisan make:request TaskRequest
```

作成された内容を確認します。

app/Http/Requests/TaskRequest.php

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class TaskRequest extends FormRequest
{
    /**
     * Determine if the user is authorized to make this request.
     *
     * @return bool
     */
    public function authorize()
    {
        return false;
    }

    /**
     * Get the validation rules that apply to the request.
     *
     * @return array
     */
    public function rules()
```

```

{
    return [
        //
    ];
}
}

```

作成したフォームリクエストは、Illuminate\Foundation\Http\FormRequest を継承したサブクラスになっています。

FormRequest を使うことができる権限があるかをチェック

authorize() メソッドで、このフォームリクエストを使用する権限があるかどうかを確認することができます。

app/Http/Requests/TaskRequest.php

```

public function authorize()
{
    return false;
}

```

今回は権限のチェックは必要ないので、true を返すようにしておきます。

```

public function authorize()
{
    return true;
}

```

バリデーションルールを記述する

rules() メソッドに、バリデーションルールを記述します。

app/Http/Requests/TaskRequest.php

```

/**
 * Get the validation rules that apply to the request.
 *
 * @return array
 */
public function rules()
{
    return [
        'title' => 'required|max:30',
        'description' => 'required|max:200',
        'expiration_date' => 'required|date',
        'completion_date' => 'nullable|date',
    ];
}

```

```
}
```

- タイトル→必須、最大 30 文字まで
- 説明→必須、最大 200 文字まで
- 期限日→必須、日付形式
- 完了日→null 許可、日付形式

登録日はアプリケーション側で生成して代入するので、バリデーションルールには含めていません。

バリデーションルールのエラーメッセージをカスタマイズする

このままだとエラーメッセージが英語になるので、日本語の適したメッセージに置き換えます。

messages()メソッドを追加します。

app/Http/Requests/TaskRequest.php

```
/**
 * バリデーションエラーのメッセージをカスタマイズする
 *
 * @return array
 */
public function messages()
{
    return [
        'title.required' => 'タイトルは必ず入力してください。',
        'title.max' => 'タイトルは 30 文字以内で入力してください。',
        'description.required' => '説明は必ず入力してください。',
        'description.max' => '説明は 200 文字以内で入力してください。',
        'expiration_date.required' => '期限日は必ず入力してください。',
        'expiration_date.date' => '期限日は正しい日付形式で入力してください。',
        'completion_date.date' => '完了日は正しい日付形式で入力してください。',
    ];
}
```

コントローラでフォームリクエストを呼び出す

コントローラで、作成したフォームリクエストを使えるようにします。

バリデーションのチェックが必要なメソッドは、

- store()
- update()

です。各メソッドの引数の型が Illuminate\Http\Request クラスになっているのを、作成したフォームリクエストの型に変更します。

まず、作成した TaskRequest クラスを使えるようにします。

app/Http/Controllers/TaskController.php

```
use App\Http\Requests\TaskRequest;
```

クラス宣言の前に上記を追記します。

store()メソッドの引数の型を下記に変更します。

```
public function store(TaskRequest $request)
```

update()メソッドの引数の型も下記に変更します。

```
public function update(TaskRequest $request, Task $task)
```

エラー表示の確認

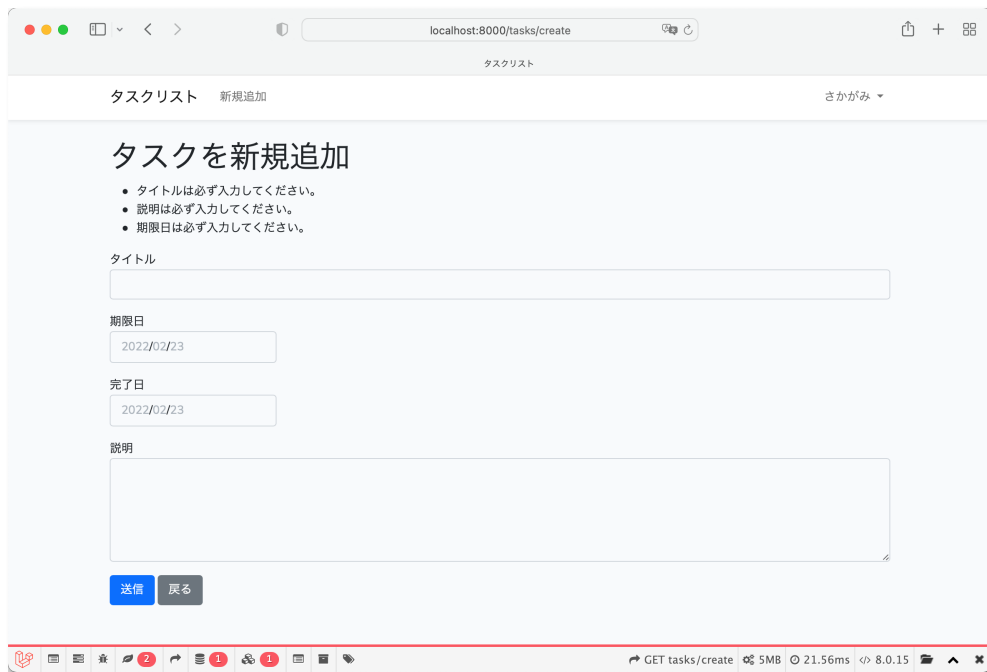
エラー表示の確認をしてみます。

create.blade.php に下記のコードを追記します。h1 タグの下あたりで OK です。

resources/views/tasks/create.blade.php

```
11. <h1>タスクを新規追加</h1>
12.
13. @if (count($errors) > 0)
14. <ul>
15.     @foreach ($errors->all() as $error)
16.         <li>{{ $error }}</li>
17.     @endforeach
18. </ul>
19. @endif
```

次のようにバリデーションエラーが表示されれば、きちんと動作しています。



エラー表示をカスタマイズする

Bootstrap を使って、エラー表示をカスタマイズしてみます。やり方はこちらのページに載っています。こちらを参考にしてみます。

<https://getbootstrap.com/docs/5.1/forms/validation/#server-side>

新規追加ページに Bootstrap のクラスを埋め込む

今は、新規追加ページのタイトルの項目の部分は、下記のようなコードになっています。

resources/views/tasks/create.blade.php

```
<div class="mb-3">
  <label for="title">タイトル</label>
  <input type="text" name="title" id="title" class="form-control" value="{{ old('title') }}">
</div>
```

ここを下記のように書き換えます。

```
<div class="mb-3">
  <label for="title">タイトル</label>
  <input type="text" name="title" id="title" class="form-control @error('title') is-invalid @enderror"
  value="{{ old('title') }}" aria-describedby="validateTitle">
  @error('title')
  <div id="validateTitle" class="invalid-feedback">
    {{ $message }}
  </div>
</div>
```

```
</div>
@enderror
</div>
```

要点は、

- バリデーションエラーがあったときに、input タグに "is-valid" クラスを追加する。
- input タグに "aria-describedby" 要素を追加し、ページ内で一意の値を指定する（上記の場合だと "validateTitle"）
- バリデーションエラーの内容を表示する div タグを用意し、id 属性に先程の一意の値（"validateTitle"）を指定し、クラス属性に "invalid-feedback" を追加する。

です。

すると、このような表示になります。

The screenshot shows a web browser window at the URL `localhost:8000/tasks/create`. The page title is "タスクリスト" (Task List) and the subtitle is "新規追加" (New Add). The main heading is "タスクを新規追加" (Add New Task). Below the heading, there are three bullet points: "タイトルは必ず入力してください。" (Title is required), "詳細は必ず入力してください。" (Details are required), and "期限日は必ず入力してください。" (Deadline is required). The form has four input fields: "タイトル" (Title), "期限日" (Deadline), "完了日" (Completion Date), and "説明" (Description). The "タイトル" field is highlighted with a red border and a red error icon, with the message "タイトルは必ず入力してください。" (Title is required) displayed below it. The "期限日" and "完了日" fields have the date "2022/02/23" entered. The "説明" field is empty. At the bottom of the form, there are two buttons: "送信" (Send) and "戻る" (Back). The browser's developer tools are open at the bottom, showing the GET request to `tasks/create`.

それぞれの入力の input タグに、同じように記載すればいいですね。

resources/views/tasks/create.blade.php

```
1. @extends('layouts.app')
2.
3. <style>
4.     input[type="date"] {
5.         width: 200px;
6.     }
7. </style>
8.
9. @section('content')
10.
11. <h1>タスクを新規追加</h1>
12.
13. {{-- @if (count($errors) > 0)
14. <ul>
```

```

15.     @foreach ($errors->all() as $error)
16.     <li>{{ $error }}</li>
17.     @endforeach
18. </ul>
19. @endif --}}
20.
21. <form action="{{ route('tasks.store') }}" method="post">
22.     @csrf
23.     <div class="mb-3">
24.         <label for="title">タイトル</label>
25.         <input type="text" name="title" id="title" class="form-control @error('title') is-invalid
@enderror "
26.             value="{{ old('title') }}" aria-describedby="validateTitle">
27.         @error('title')
28.         <div id="validateTitle" class="invalid-feedback">
29.             {{ $message }}
30.         </div>
31.         @enderror
32.     </div>
33.     <div class="mb-3">
34.         <label for="expiration_date">期限日</label>
35.         <input type="date" name="expiration_date" id="expiration_date"
36.             class="form-control @error('expiration_date') is-invalid @enderror "
value="{{ old('expiration_date') }}"
37.             aria-describedby="validateExpirationDate">
38.         @error('expiration_date')
39.         <div id="validateExpirationDate" class="invalid-feedback">
40.             {{ $message }}
41.         </div>
42.         @enderror
43.     </div>
44.     <div class="mb-3">
45.         <label for="completion_date">完了日</label>
46.         <input type="date" name="completion_date" id="completion_date"
47.             class="form-control @error('completion_date') is-invalid @enderror "
value="{{ old('completion_date') }}"
48.             aria-describedby="validateCompletionDate">
49.         @error('completion_date')
50.         <div id="validateCompletionDate" class="invalid-feedback">
51.             {{ $message }}
52.         </div>
53.         @enderror
54.     </div>
55.     <div class="mb-3">
56.         <label for="description">説明</label>
57.         <textarea name="description" id="descriptin" rows="5"
58.             class="form-control @error('description') is-invalid @enderror "
59.             aria-describedby="validateDescription">{{ old('description') }}</textarea>
60.         @error('description')
61.         <div id="validateDescription" class="invalid-feedback">
62.             {{ $message }}
63.         </div>
64.         @enderror
65.     </div>
66.     <button class="btn btn-primary" type="submit">送信</button>
67.     <a href="{{ route('tasks.index') }}" class="btn btn-secondary">戻る</a>
68. </form>
69.

```

70. @endsection
71.

ブラウザで表示したときに、このように表示されれば OK です。

タスクリスト 新規追加 さかがみ ▼

タスクを新規追加

タイトル

タイトルは必ず入力してください。

期限日

2022/02/23

期限日は必ず入力してください。

完了日

2022/02/23

説明

説明は必ず入力してください。

送信 戻る

GET tasks/create 5MB 14.39ms 8.0.15

日付形式の入力項目のバリデーションチェックのテストができない

<input type="date">など、テキスト形式以外の入力項目は、その形式でしか入力できなくなっているの、誤った形式のデータを入力してバリデーションチェックのテストができません。

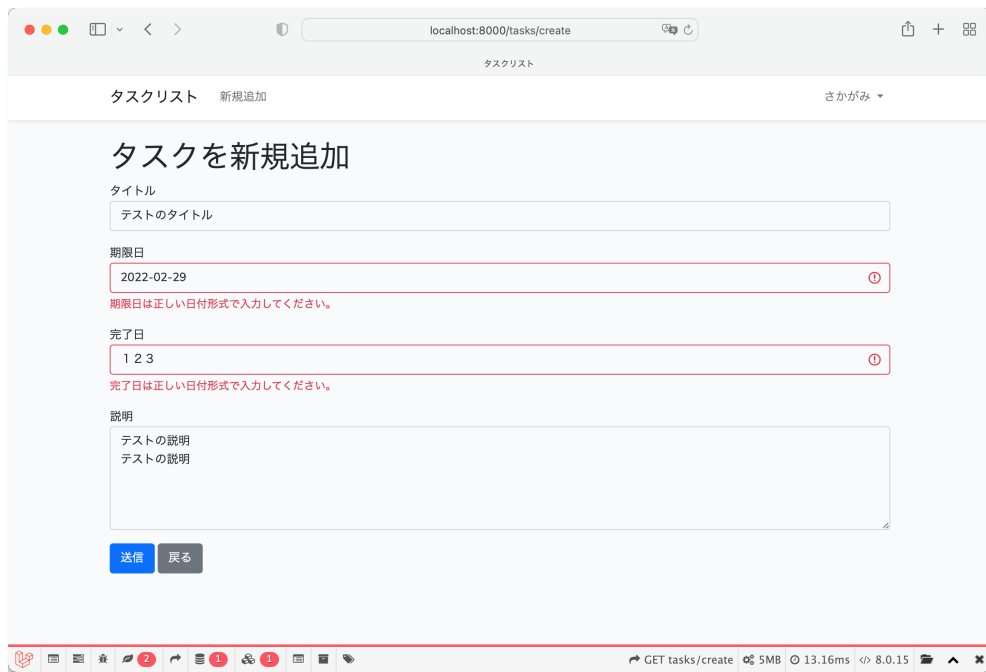
その場合は、<input type="text">にして、文字列の形式にしてからテストしましょう。

```
<input type="date" name="expiration_date" id="expiration_date"
```

を

```
<input type="text" name="expiration_date" id="expiration_date"
```

に修正してテストを行います。



修正ページにも Bootstrap のクラスを埋め込む

修正ページにも、新規追加ページと同じように Bootstrap のクラスを埋め込んでいきます。

ただし、修正ページの入力項目のデフォルト値は、

- バリデーションエラーがないときは、モデルが持っている値
- バリデーションエラーがあるときは、元の入力値

にする必要があります。下記のように記述します。old()は第2引数にデフォルト値を設定できます。モデルのプロパティの値をデフォルト値として設定します。

```
<input type="text" name="title" id="title" class="form-control" value="{{ old('title', $task->title) }}">
```

すべての項目に埋め込んでみたソースです。

resources/views/tasks/edit.blade.php

```
1. @extends('layouts.app')
2.
3. <style>
4.     input[type="date"] {
5.         width: 200px;
6.     }
7. </style>
8.
9. @section('content')
10.
11. <h1>タスクを修正</h1>
12. <form action="{{ route('tasks.update', $task) }}" method="post">
13.     @csrf
14.     @method('PATCH')
```

```

15.     <div class="mb-3">
16.         <label for="title">タイトル</label>
17.         <input type="text" name="title" id="title" class="form-control @error('title') is-invalid
@enderror "
18.             value="{{ old('title', $task->title) }}">
19.         @error('title')
20.         <div id="validateTitle" class="invalid-feedback">
21.             {{ $message }}
22.         </div>
23.         @enderror
24.     </div>
25.     <div class="mb-3">
26.         <label for="expiration_date">期限日</label>
27.         <input type="date" name="expiration_date" id="expiration_date"
28.             class="form-control @error('expiration_date') is-invalid @enderror "
29.             value="{{ old('expiration_date', $task->expiration_date->format("Y-m-d")) }}">
30.         @error('expiration_date')
31.         <div id="validateExpirationDate" class="invalid-feedback">
32.             {{ $message }}
33.         </div>
34.         @enderror
35.     </div>
36.     <div class="mb-3">
37.         <label for="completion_date">完了日</label>
38.         <input type="date" name="completion_date" id="completion_date"
39.             class="form-control @error('completion_date') is-invalid @enderror "
40.             value="{{ old('completion_date' , is_null($task->completion_date) ? "" : $task-
>completion_date->format("Y-m-d")) }}">
41.         @error('completion_date')
42.         <div id="validateCompletionDate" class="invalid-feedback">
43.             {{ $message }}
44.         </div>
45.         @enderror
46.     </div>
47.     <div class="mb-3">
48.         <label for="description">説明</label>
49.         <textarea name="description" id="descriptin" rows="5"
50.             class="form-control @error('description') is-invalid @enderror ">{{ old('description',
$task->description) }}</textarea>
51.         @error('description')
52.         <div id="validateDescription" class="invalid-feedback">
53.             {{ $message }}
54.         </div>
55.         @enderror
56.     </div>
57.     <button class="btn btn-primary" type="submit">送信</button>
58.     <a href="{{ route('tasks.index') }}" class="btn btn-secondary">戻る</a>
59. </form>
60.
61. @endsection
62.

```

次のように表示されれば OK です。

